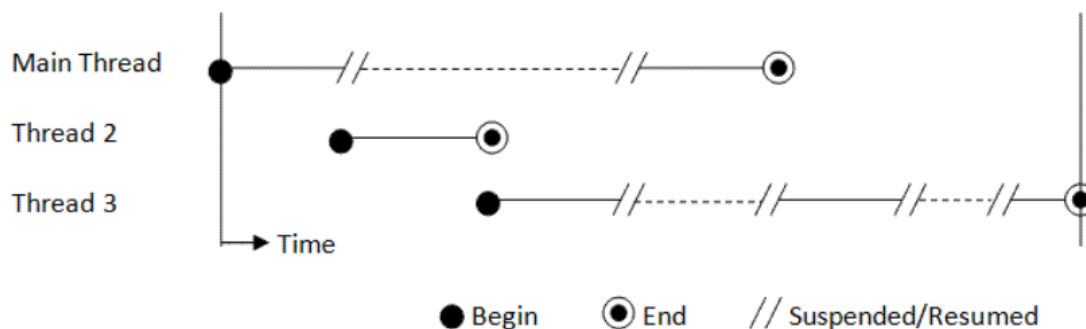


Programmazione Concorrente (multithread)

La programmazione concorrente (parallela) consiste nello sfruttare l'architettura multicore dei processori, permettendo di ottimizzare l'esecuzione di operazioni onerose suddividendole in sottoprocessi eseguiti parallelamente. Si evita, in tal modo, il blocco di risorse nell'attesa del loro completamento. (stampa, copia di file, caricamento dati, server, GUI...)

I **thread** non sono altro che sotto-processi eseguiti generalmente in parallelo, e lanciati da un processo padre.

Va notato che attualmente in Python non è possibile eseguire in parallelo più thread, anche se sono eseguiti su più core, soltanto uno per volta viene avviato.



Processo: sequenza di attività (Programma) eseguite in RAM. Composto da un `mainThread`, che a sua volta si suddividerà in thread secondari `workerThreads`.

Thread: suddivisione di un processo in due o più sottoprocessi eseguiti in modo concorrente

L'esecuzione parallela di più thread/processi può generare problemi di **sincronizzazione**, come ad esempio l'accesso contemporaneo ad una risorsa.

Tipico è il **deadlock**: due thread/processi stanno cercando di acquisire due risorse diverse, ma uno ha già acquisito la prima risorsa e sta aspettando la seconda, mentre l'altro ha acquisito la seconda risorsa e sta aspettando la prima. Nessuno dei thread può continuare l'esecuzione, poichè entrambi sono in attesa che l'altra risorsa venga rilasciata.

Altro problema è la **starvation**: un thread/processo con bassa priorità rischia di non venir mai eseguito.

Esistono appositi algoritmi di **scheduling** in grado di ovviare principalmente i problemi di starvation, ma per il deadlock conviene utilizzare accorgimenti durante lo sviluppo multithreading (lock, semaphore)

Tipico il [Problema dei filosofi a cena](#)

Lock: oggetto che un thread deve "bloccare in modo esclusivo" prima di poter eseguire una sezione critica di un programma (accesso ad una risorsa condivisa)

Semaphore: contatore utilizzato per limitare il numero di thread che possono accedere contemporaneamente ad una sezione critica di un programma.

Creazione ed esecuzione di un semplice thread in Python

```
import threading
import time

def myThread():
    cont = 0
    print("Il thread è in esecuzione")
    current_thread = threading.current_thread()
    current_thread.name = "Primo Thread"
    print(f"Name -> {current_thread.name}")
    while cont < 10:
        print(f"Thread -> {cont}")
        cont += 1
        time.sleep(0.5) # pausa di 500 millisecondi
    print("Il thread è terminato")

# Istanzio il thread
mioThread = threading.Thread(target=myThread)
print("Thread Principale: avvio mioThread")
mioThread.start()

# Il metodo join() arresta l'esecuzione del thread padre fino al termine del thread figlio

mioThread.join()
print("Thread Principale: mioThread è terminato")
```

```
C:\Users\admin\Desktop\Python\Thread>python Es01_Primo_Thread.py
Thread Principale: avvio mioThread
Il thread è in esecuzione
Name -> Primo Thread
Thread -> 0
Thread -> 1
Thread -> 2
Thread -> 3
Thread -> 4
Thread -> 5
Thread -> 6
Thread -> 7
Thread -> 8
Thread -> 9
Il thread è terminato
Thread Principale: mioThread è terminato
```

Utilizzo di Lock in Python

`lock.acquire()` blocca la risorsa (area critica) per esclusivo uso del thread

`lock.release()` rilascia la risorsa (area critica)

Analogo ad utilizzare:

```
with lock:
    #istruzioni area critica
```

```
import threading
import random
import time

def threadLock():
    nome = threading.current_thread().name
    for i in range(1, 6):
        lock.acquire() #blocco la risorsa (area critica)
        s = rnd.randint(1, 3)
        time.sleep(s)
        print(f"{nome} - Numero: {i}")
        lock.release() #rilascio la risorsa (area critica)

#-----MAIN
rnd = random.Random()
# Creo il lock
lock = threading.Lock()
# Creo i thread
thread1 = threading.Thread(target=threadLock)
thread1.name = "Thread-1"
thread2 = threading.Thread(target=threadLock)
thread2.name = "Thread-2"
# Avvio i thread
thread1.start()
thread2.start()
# Attendo che i thread terminino
thread1.join()
thread2.join()
print("Esecuzione completata!")
```

```
C:\Users\admin\Desktop\Python\Thread>python thread_lock.py
Thread-1 - Numero: 1
Thread-1 - Numero: 2
Thread-1 - Numero: 3
Thread-1 - Numero: 4
Thread-2 - Numero: 1
Thread-2 - Numero: 2
Thread-2 - Numero: 3
Thread-2 - Numero: 4
Thread-1 - Numero: 5
Thread-2 - Numero: 5
Esecuzione completata!
```

Utilizzo di Semaphore in Python

```
semaphore = threading.Semaphore(n,max)
```

n: valore iniziale del contatore del semaforo, definisce il numero di thread che possono accedere simultaneamente alla sezione critica. Se il contatore è zero, nessun thread può accedere fino a quando un altro thread non chiama `release()`.

max: (Opzionale) numero massimo di risorse contemporanee che il semaforo può ammettere, ossia il limite superiore del contatore del semaforo.

Funzionamento del Semaforo:

acquire(): ogni volta che un thread esegue `acquire()`, il semaforo decrementa il contatore. Quando il contatore è maggiore di zero, il thread può procedere, mentre se è zero, il thread rimane in attesa fino a quando un altro thread non incrementa il contatore eseguendo `release()`.

release(): ogni volta che un thread esegue `release()` il semaforo incrementa il contatore. Se ci sono thread bloccati in attesa, uno di essi verrà svegliato e potrà procedere.

```
import threading
import random
import time

def threadSemaphore():
    nome = threading.current_thread().name
    for i in range(1, 3):
        with semaphore: #equivale a usare semaphore.acquire() e semaphore.release()
            s = rnd.randint(1, 3)
            time.sleep(s)
            print(f'Thread {nome} numero: {i}')

#-----MAIN
rnd = random.Random()
# creo il semaphore
semaphore = threading.Semaphore(4)
# creo una lista thread e li avvio
threads = []
for i in range(10):
    th = threading.Thread(target=threadSemaphore)
    th.name = f"Thread-{i}"
    threads.append(th)
    th.start()
# Attendo che i thread terminino
for i in range(10):
    threads[i].join()
print("Esecuzione completata!")
```

```
C:\Users\admin\Desktop\Python\Thread>python thread_semaphore.py
Thread Thread-1 numero: 1
Thread Thread-3 numero: 1
Thread Thread-3 numero: 2
Thread Thread-0 numero: 1
Thread Thread-1 numero: 2
Thread Thread-2 numero: 1
Thread Thread-2 numero: 2
Thread Thread-6 numero: 1
Thread Thread-0 numero: 2
Thread Thread-8 numero: 1
Thread Thread-7 numero: 1
Thread Thread-7 numero: 2
Thread Thread-4 numero: 1
Thread Thread-6 numero: 2
Thread Thread-8 numero: 2
Thread Thread-4 numero: 2
Thread Thread-5 numero: 1
Thread Thread-5 numero: 2
Thread Thread-9 numero: 1
Thread Thread-9 numero: 2
Esecuzione completata!
```